



第十讲 存储过程、触发器和游标



教学内容

- **10.1 存储过程**
 - 存储过程介绍
 - 创建存储过程
 - 使用存储过程
- **10.2 触发器**
 - 触发器介绍
 - 创建触发器
 - 管理触发器
- **10.3 游标**
 - 游标介绍
 - 创建游标
 - 使用游标



10.1.1 存储过程概念

- 存储过程是将SQL语句放入一个集合里，然后直接调用存储过程来执行已经定义好的SQL语句集合
- 避免开发人员重复编写相同的SQL语句
- 存储过程还可以减少数据在数据库和应用服务器之间的传输，可以提高数据处理的效率。



存储过程的优点

- 存储过程中可以包含数据存取语句、流程控制语句、错误处理语句等，在使用上非常有弹性。
- 优点
 - 执行效率高
 - 统一的操作流程
 - 重用性、共享性和可移植性
 - 安全性高



10.1.2 创建存储过程

```
CREATE  [DEFINER = { user | CURRENT_USER }]
        PROCEDURE sp_name ([proc_parameter[,...]])
        [characteristic ...] routine_body
proc_parameter:  [ IN | OUT | INOUT ] param_name type
characteristic:  COMMENT 'string' | LANGUAGE SQL
                | [NOT] DETERMINISTIC | { CONTAINS SQL | NO SQL | READS SQL DATA |
MODIFIES SQL DATA } | SQL SECURITY { DEFINER | INVOKER }
routine_body:
        Valid SQL routine statement
[begin_label:] BEGIN
        [statement_list]
        .....
END [end_label]
```



存储过程中的关键语法

- 声明语句结束符:
 - DELIMITER \$\$ 或 DELIMITER //
- 声明存储过程:
 - CREATE PROCEDURE demo_in_parameter(IN p_in int)
- 存储过程开始和结束符号:
 - BEGIN END
- 变量赋值:
 - SET @p_in=1
- 变量定义:
 - DECLARE l_int int unsigned default 4000000;



10.1.3 存储过程的参数

- **CREATE PROCEDURE** 存储过程名([**IN** | **OUT** | **INOUT**] 参数名 数据类型...)]
- 三种参数类型
 - **IN** 输入参数：表示调用者向过程传入值（传入值可以是字面量或变量）
 - **OUT** 输出参数：表示过程向调用者传出值(可以返回多个值)（传出值只能是变量）
 - **INOUT** 输入输出参数：既表示调用者向过程传入值，又表示过程向调用者传出值（值只能是变量）
- 查看存储过程的定义
 - `show create procedure proc_name;`



in 输入参数

```
delimiter $$  
create procedure in_param(in p_in int)  
begin  
    select p_in;  
    set p_in=2;  
    select P_in;  
end $$  
delimiter ;  
set @p_in=1;  
call in_param(@p_in);  
select @p_in;
```




out输出参数

```
delimiter //  
  
create procedure out_param(out p_out int)  
begin  
    select p_out;  
    set p_out=2;  
    select p_out;  
end //  
  
delimiter ;  
  
set @p_out=1;  
call out_param(@p_out);  
select @p_out;
```



inout输入参数

```
delimiter $$
```

```
create procedure inout_param(inout p_inout int)
```

```
begin
```

```
select p_inout;
```

```
set p_inout=2;
```

```
select p_inout;
```

```
end $$
```

```
delimiter ;
```

```
set @p_inout=1;
```

```
call inout_param(@p_inout);
```

```
select @p_inout;
```



inout输入参数

```
DELIMITER //  
CREATE PROCEDURE sp_test(IN p_num INT,OUT p_result INT)  
BEGIN  
  SET p_result = 1;  
  WHILE p_num>1 DO  
    SET p_result = p_result * p_num ;  
    SET p_num = p_num -1;  
  END WHILE;  
END //  
DELIMITER ;  
CALL sp_test(5,@result);  
SELECT @result;
```



求阶乘存储过程

```
create procedure get_jc(in n int)
begin
    declare i int default 1;
    declare k int default 1;
    while i<=n do
        set k=k*i;
        set i=i+1;
    end while;
    select i;
    select k;
end;
```



Case存储过程

```
DELIMITER //  
CREATE PROCEDURE proc3 (in parameter int)  
begin  
    declare var int;  
    set var=parameter+1;  
    case var  
    when 0 then  
        insert into t values(17);  
    when 1 then  
        insert into t values(18);  
    else  
        insert into t values(19);  
    end case;  
end; //  
DELIMITER ;
```



While存储过程添加记录

```
DELIMITER //  
CREATE PROCEDURE proc4()  
    begin  
        declare var int;  
        set var=0;  
        while var<6 do  
            insert into t values(var);  
            set var=var+1;  
        end while;  
    end;  
//  
DELIMITER ;
```



学生存储过程

```
CREATE PROCEDURE sp_stud (IN p_num INT)  
BEGIN  
    CASE p_num  
        WHEN 1 THEN  
            SELECT * FROM StudInfo WHERE StudName LIKE '王%';  
        WHEN 2 THEN  
            SELECT * FROM StudInfo WHERE StudName LIKE '张%';  
        ELSE  
            SELECT * FROM StudInfo WHERE StudName LIKE '李%';  
    END CASE;  
END ;
```



存储过程示例

```
DELIMITER //  
DROP PROCEDURE if exists P_Save_ClassInfo;  
Create Procedure P_Save_ClassInfo  
(in CID Varchar(10),in CName Varchar(50),in CDesc Varchar(100))  
begin  
  if Exists(Select * From ClassInfo Where ClassID=CID) then  
    Update ClassInfo Set ClassName=CName,ClassDesc=CDesc  
      Where ClassID=CID;  
  
  Else  
    Insert Into ClassInfo(ClassID,ClassName,ClassDesc) Values  
(CID,CName,CDesc);  
End if;  
End //  
DELIMITER ;
```


创建班级信息保存存储过程，如果班级存在就更新，不存在就添加，并给出更新成功或保存成功提示。

```

DELIMITER //
DROP PROCEDURE if exists P_Save_ClassInfo;
Create Procedure P_Save_ClassInfo(in CID Varchar(10),in CName Varchar(50),
                                in CDesc Varchar(100), [填空1] msg varchar(20))
begin
    if [填空2](Select * From ClassInfo Where ClassID=CID) then
        Update ClassInfo Set ClassName=CName,ClassDesc=CDesc Where ClassID=[填空3];
        set msg='修改成功';
    Else
        Insert Into ClassInfo(ClassID,ClassName,ClassDesc) Values (CID,CName,CDesc);
        set [填空4]='添加成功';
    End if;
End //
DELIMITER ;

call P_Save_ClassInfo('20231152','computer2023','good',@msg);
select @msg;
    
```

作答



存储过程示例

```
DELIMITER //  
DROP PROCEDURE if exists P_Save_ClassInfo;  
Create Procedure P_Save_ClassInfo  
(in CID Varchar(10),in CName Varchar(50),in CDesc Varchar(100),out msg varchar(20))  
begin  
  if Exists(Select * From ClassInfo Where ClassID=CID) then  
    Update ClassInfo Set ClassName=CName,ClassDesc=CDesc  
      Where ClassID=CID;  
    set msg='修改成功';  
  
  Else  
    Insert Into ClassInfo(ClassID,ClassName,ClassDesc) Values (CID,CName,CDesc);  
    set msg='添加成功';  
  
  End if;  
  End //  
DELIMITER ;  
call P_Save_ClassInfo('20231152','computer2023','good',@msg);  
select @msg;
```



数据库原理与应用

第十讲 存储过程、触发器和游标

10.2 触发器

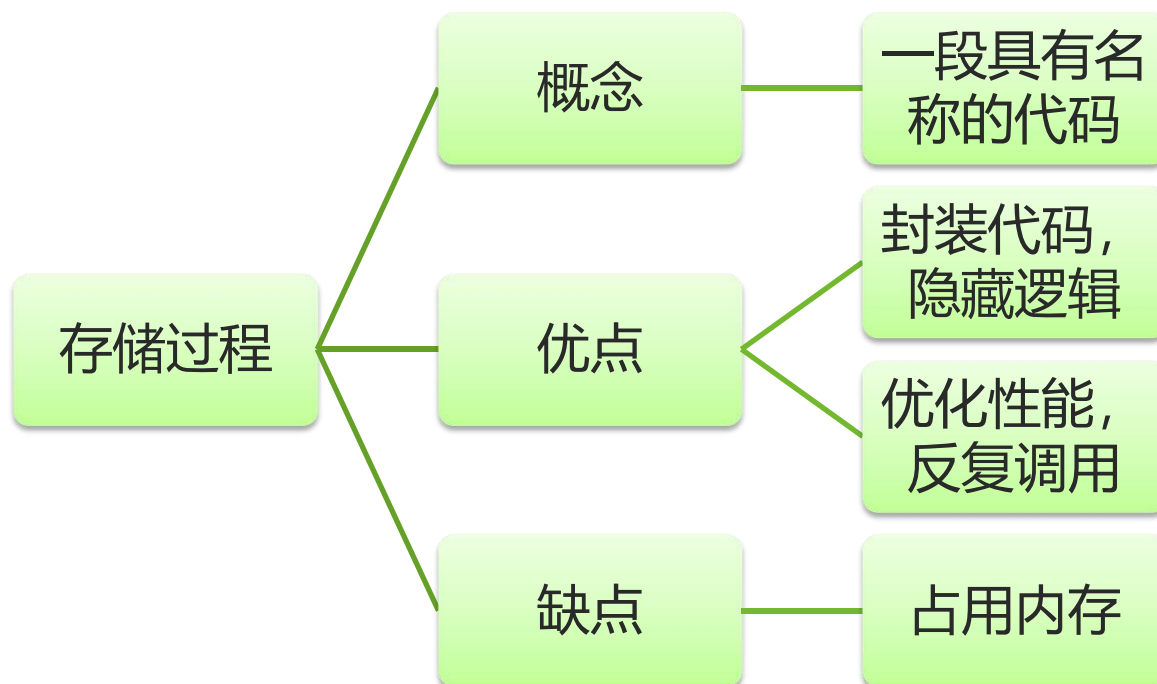


存储过程回顾

• 语法

- CREATE PROCEDURE proc_name
(IN | OUT | INOUT parameters
DATATYPE)
- BEGIN
- -- 变量定义
- -- 变量赋值
- -- 过程处理
- END;

• 功能





存储过程示例

```
DELIMITER //
DROP PROCEDURE IF EXISTS P_Save_ClassInfo;
CREATE PROCEDURE P_Save_ClassInfo
(IN CID VARCHAR(10),IN CName VARCHAR(50),IN CDesc VARCHAR(100),OUT msg
VARCHAR(20))
BEGIN
  IF EXISTS(SELECT * FROM ClassInfo WHERE ClassID=CID) THEN
    UPDATE ClassInfo SET ClassName=CName,ClassDesc=CDesc WHERE ClassID=CID;
    SET msg='修改成功';
  Else
    Insert Into ClassInfo(ClassID,ClassName,ClassDesc) Values (CID,CName,CDesc);
    SET msg='添加成功';
  END IF;
END //
DELIMITER ;
```

```
CALL P_Save_ClassInfo('20231152','computer2023','good',@msg);
SELECT @msg;
```



你用过触发器吗？



What is 触发器？





学生选课案例

教师任课表（TeacherCourseInfo）

字段名称	数据类型	字段长度	PK	字段描述	举例
TeacherNo	VARCHAR	10	Y	教师编号	HB001
CourseID	VARCHAR	15	Y	课程编号	GDSX01
LimitPersonCount	INT			限选人数	50
RemainPersonCount	INT			已选人数	2

选课结果表（ElectCourseResult）

字段名称	数据类型	字段长度	PK	字段描述	举例
StudNo	VARCHAR	15	Y	学生学号	20231152001
TeacherNo	VARCHAR	10	Y	教师编号	HB001
CourseID	VARCHAR	15	Y	课程编号	GDSX01



创建教师任课表 (TeacherCourseInfo)

```
CREATE TABLE TeacherCourseInfo  
(  
    TeacherNo VARCHAR(10),  
    CourseID VARCHAR(15),  
    LimitPersonCount INT,  
    RemainPersonCount INT,  
    CONSTRAINT PK_T_C PRIMARY KEY (TeacherNo, CourseID)  
);
```




添加测试数据

- **INSERT INTO TeacherCourseInfo**
(TeacherNo,CourseID,LimitPersonCount,RemainPersonCount)
VALUES
(**'HB001'**, **'GDSX01'**, **50**, **0**);
- **INSERT INTO TeacherCourseInfo**
VALUES (**'LL002'**, **'GDSX01'**, **60**, **0**);
- **INSERT INTO TeacherCourseInfo**
VALUES (**'HB001'**, **'GCSX03'**, **70**, **0**);
- **INSERT INTO TeacherCourseInfo**
VALUES (**'LL002'**, **'GCSX03'**, **80**, **0**);



创建选课结果表 (ElectCourseResult)

```
CREATE TABLE ElectCourseResult  
(  
    StudNo VARCHAR(15),  
    TeacherNo VARCHAR(10),  
    CourseID VARCHAR(15),  
    CONSTRAINT PK_S_T_C PRIMARY KEY  
        (StudNo, TeacherNo, CourseID)  
);
```



学生选课

```
INSERT INTO ElectCourseResult
VALUES
('20231152001','HB001','GDSX01');
```

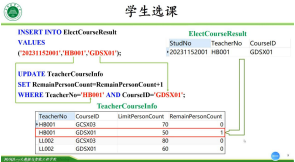
```
UPDATE TeacherCourseInfo
SET RemainPersonCount=RemainPersonCount+1
WHERE TeacherNo='HB001' AND CourseID='GDSX01';
```

ElectCourseResult

StudNo	TeacherNo	CourseID

TeacherCourseInfo

TeacherNo	CourseID	LimitPersonCount	RemainPersonCount
HB001	GCSX03	70	0
HB001	GDSX01	50	0
LL002	GCSX03	80	0
LL002	GDSX01	60	0





学生选课

```
INSERT INTO ElectCourseResult  
VALUES  
('20231152001','HB001','GDSX01');
```

ElectCourseResult

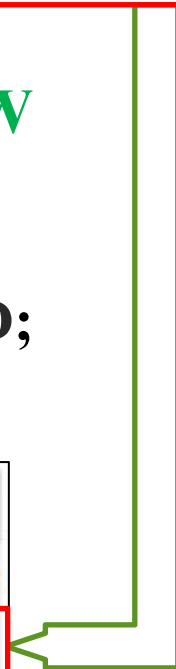
StudNo	TeacherNo	CourseID
▶ 20231152001	HB001	GDSX01

↑
NEW

```
UPDATE TeacherCourseInfo  
SET RemainPersonCount=RemainPersonCount+1  
WHERE TeacherNo=NEW.TeacherNo AND CourseID=NEW.CourseID;
```

TeacherCourseInfo

TeacherNo	CourseID	LimitPersonCount	RemainPersonCount
▶ HB001	GCSX03	70	0
HB001	GDSX01	50	1
LL002	GCSX03	80	0
LL002	GDSX01	60	0





创建学生选课触发器 (INSERT)

DELIMITER //

CREATE TRIGGER T_ElectCourseResult_INSERT

AFTER INSERT

ON ElectCourseResult

FOR EACH ROW

BEGIN

UPDATE TeacherCourseInfo

SET RemainPersonCount=RemainPersonCount+1

WHERE TeacherNo=**NEW**.TeacherNo

AND CourseID=**NEW**.CourseID;

END //

DELIMITER ;



测试触发器

INSERT INTO ElectCourseResult
VALUES

('20231152005', 'LL002', 'GDSX01'); ➡

ElectCourseResult

StudNo	TeacherNo	CourseID
20231152001	HB001	GDSX01



触发器

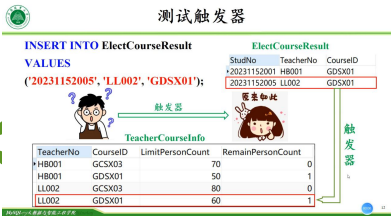


TeacherCourseInfo

UPDATE TeacherCourseInfo
SET RemainPersonCount=RemainPersonCount+1
WHERE TeacherNo=NEW.TeacherNo
AND CourseID=NEW.CourseID;

TeacherCourseInfo			
			RemainPersonCount
70			0
50			1
30			0
60	LL002	GDSX01	0

触发器





触发器 (TRIGGER)

- 概念

- 触发器是针对单一数据表所撰写的特殊存储过程，当该数据表发生INSERT、UPDATE或DELETE时会自动被触发(执行)。

- 语法

- **CREATE TRIGGER** trigger name
 - {**BEFORE** | **AFTER**} {**INSERT** | **UPDATE** | **DELETE**}
 - **ON** table_name
 - **FOR EACH ROW**
 - **BEGIN**
 - trigger_body; -- 触发器语句主体，每行用分号结尾
 - **END;**

触发事件

触发时间



选课触发器 (INSERT) 回头看

DELIMITER //

CREATE TRIGGER T_ElectCourseResult_INSERT

AFTER INSERT

ON ElectCourseResult

FOR EACH ROW

BEGIN

UPDATE TeacherCourseInfo

SET RemainPersonCount=RemainPersonCount+1

WHERE TeacherNo=**NEW**.TeacherNo

AND CourseID=**NEW**.CourseID;

END //

DELIMITER ;



测试触发器

```
INSERT INTO ElectCourseResult
VALUES
('20231152005', 'LL002', 'GDSX01');
```

ElectCourseResult

StudNo	TeacherNo	CourseID
20231152001	HB001	GDSX01
20231152005	LL002	GDSX01

学生一直选一位老师的一门课程



触发器

TeacherNo	CourseID	LimitPersonCount	RemainPersonCount
HB001	GCSX03	70	0
HB001	GDSX01	50	1
LL002	GCSX03	80	0
LL002	GDSX01	60	1



存储过程

- DELIMITER //
- CREATE PROCEDURE Proc_Elect_Course
- (IN SNO VARCHAR(15),IN TNO VARCHAR(10),IN CNO VARCHAR(15),OUT msg VARCHAR(20))
- BEGIN
- IF EXISTS(SELECT * FROM ElectCourseResult WHERE StudNo=SNO AND TeacherNo=TNO AND CourseID=CNO) THEN
- SET msg='学生已选此课';
- ELSE
- IF EXISTS(SELECT * FROM TeacherCourseInfo WHERE TeacherNo=TNO AND CourseID=CNO AND RemainPersonCount<LimitPersonCount) THEN
- INSERT INTO ElectCourseResult VALUES (SNO,TNO,CNO);
- SET msg='选课成功';
- ELSE
- SET msg='选课失败';
- END IF;
- END IF ;
- END //
- DELIMITER ;

```
CALL Proc_Elect_Course('20231152001','HB001',  
'GDSX01',@msg);  
SELECT @msg;
```





触发器中的NEW & OLD

- **NEW**

- 存储将要插入的记录
- INSERT型触发器
- **NEW**.columnName

- **OLD**

- 存储将要删除的记录
- DELETE型触发器
- **OLD**.columnName





我要退课



退课！退课！退课！



我后悔





退课触发器

DELETE FROM ElectCourseResult

WHERE StudNo='20231152005'

AND TeacherNo= 'LL002'

AND CourseID= 'GDSX01';

UPDATE TeacherCourseInfo

SET RemainPersonCount=RemainPersonCount-1

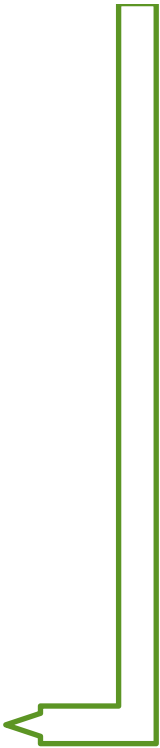
WHERE TeacherNo=OLD.TeacherNo **AND** CourseID=OLD.CourseID;

TeacherCourseInfo

ElectCourseResult

StudNo	TeacherNo	CourseID
20231152001	HB001	GDSX01

TeacherNo	CourseID	LimitPersonCount	RemainPersonCount
HB001	GCSX03	70	0
HB001	GDSX01	50	1
LL002	GCSX03	80	0
LL002	GDSX01	60	0



创建退课触发器，学生退课，教师对应的已选课程人数减1。

```
DELIMITER //
CREATE TRIGGER T_ElectCourseResult_DELETE
AFTER [填空1]
ON ElectCourseResult
FOR EACH ROW
BEGIN
    UPDATE TeacherCourseInfo
        SET RemainPersonCount=RemainPersonCount [填空2]
        WHERE TeacherNo=[填空3]
            AND CourseID=[填空4];
END //
DELIMITER ;
```

ElectCourseResult

StudNo	TeacherNo	CourseID
20231152001	HB001	GDSX01
20231152005	LL002	GDSX01

TeacherCourseInfo

TeacherNo	CourseID	LimitPersonCount	RemainPersonCount
HB001	GCSX03	70	0
HB001	GDSX01	50	1
LL002	GCSX03	80	0
LL002	GDSX01	60	1

作答



创建学生退课触发器 (DELETE)

DELIMITER //

CREATE TRIGGER T_ElectCourseResult_DELETE

AFTER DELETE

ON ElectCourseResult

FOR EACH ROW

BEGIN

UPDATE TeacherCourseInfo

SET RemainPersonCount=RemainPersonCount - 1

WHERE TeacherNo=OLD.TeacherNo

AND CourseID=OLD.CourseID;

END //

DELIMITER ;

-- 退课测试

DELETE FROM ElectCourseResult

WHERE StudNo='20231152005'

AND TeacherNo= 'LL002'

AND CourseID= 'GDSX01';

创建学生退课触发器 (DELETE)

```
DELIMITER //
CREATE TRIGGER T_ElectCourseResult_DELETE
AFTER DELETE
ON ElectCourseResult
FOR EACH ROW
BEGIN
    UPDATE TeacherCourseInfo
    SET RemainPersonCount=RemainPersonCount - 1
    WHERE TeacherNo=OLD.TeacherNo
    AND CourseID=OLD.CourseID;
END //
DELIMITER ;
```



触发器作用

- 检查所做的更改是否允许
- 进行其他相关数据的更改动作（**级联更改**）
- 发出更改或警告通知
- 自定义错误信息
- 更改**原**来所要进行的**数据**操作



更改原数据

```
CREATE TABLE Goods(  
    Price DOUBLE,          -- 单价  
    Quantity DOUBLE,       -- 数量  
    Total DOUBLE NULL      -- 总计  
);
```

```
DELIMITER //
```

```
CREATE TRIGGER T_Goods_INSERT  
BEFORE INSERT ON Goods  
FOR EACH ROW  
BEGIN
```

```
    SET NEW.Total = NEW.Price*NEW.Quantity;
```

```
END //
```

```
DELIMITER ;
```

```
INSERT INTO Goods  
(Price,Quantity,Total)  
VALUES  
(20, 5, 20);
```



Goods

Price	Quantity	Total
20	5	100





查看和删除触发器

- 查看触发器

- 查看全部触发器

- `SHOW TRIGGERS;`

- 查看触发器的创建语句

- `SHOW CREATE TRIGGER trigger_name;`
 - 例: `SHOW CREATE TRIGGER T_ElectCourseResult_INSERT;`

```
1 CREATE DEFINER=`root`@`localhost` TRIGGER `T_ElectCourseResult_INSERT`  
  AFTER INSERT ON `electcourseresult` FOR EACH ROW BEGIN  
2 UPDATE TeacherCourseInfo  
3 SET RemainPersonCount=RemainPersonCount+1  
4 WHERE TeacherNo=NEW.TeacherNo  
5       AND CourseID=NEW.CourseID;  
6 END
```



- 删除触发器

- `DROP TRIGGER trigger_name;`

- 例: `DROP TRIGGER T_ElectCourseResult_INSERT;`



小 结

语法

```
CREATE TRIGGER trigger_name  
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
ON table_name  
FOR EACH ROW  
BEGIN  
    trigger_body; -- 触发器语句主体，每行用分号结尾  
END;
```

功能

- 级联修改数据
- 更改原数据

重要

- **NEW**
- **OLD**



行业应用：级联更新

FL_SYS_ZYSJZD

L_ID	L_PARID	L_JB	L_BH	C_DOMAINNAME	C_DOMAINDE	C_CODE	C_NAME
38	35	2	3	地貌	地貌	3	中山
39	35	2	4	地貌	地貌	4	低山
40	35	2	5	地貌	地貌	5	丘陵
41	35	2	6	地貌	地貌	6	平原
42	0	1	3	坡向	坡向	坡向	坡向
43	42	2	1	坡向	坡向	1	北
44	42	2	2	坡向	坡向	2	东北
45	42	2	3	坡向	坡向	3	东

管理一张图
系统

LDGX_P

PK_UID	SHENG	XIAN	XIANG	CUN	LIN_YE_JU	LIN_CHANG	LIN_BAN	XIAO_BAN	DI_MAO	PO_XIANG	PO_WEI	PO_DU											
													Click here to define a filter										
6	53	...	530426	...	007	...	004	...	0.0	...	(null)	...	0061	...	00035	...	3	...	5	...	2	...	2
7	53		530426		007		004		0.0		(null)		0069		00038		3		4		2		3
8	53		530426		007		004		0.0		(null)		0068		00001		3		1		3		3
9	53		530426		007		004		15.9		(null)		0063		00029		3		5		5		3
10	53		530426		007		004		0.0		(null)		0064		00032		3		5		2		3
11	53		530426		007		004		0.0		(null)		0061		00016		3		3		3		2
12	53		530426		007		004		0.0		(null)		0065		00018		3		5		1		3
13	53		530426		007		004		0.0		(null)		0056		00029		3		4		4		3
14	53		530426		007		004		0.0		(null)		0055		00037		3		3		5		4
15	53		530426		007		004		0.0		(null)		0055		00033		3		4		2		3

初始化: MapZone



思考 & 作业

- 班级信息表人数更新
ClassInfo

ClassID	ClassName	ClassDesc
20230505	农经2023	Good
20230708	林学2023	Good
20230709	法学2023	Good
20230711	工商2023	Good

StudInfo

StudNo	StudName	StudGender	StudBirthDay	ClassID
20230505001	董晓燕	男	2004-10-16	20230505
20230505002	刘惠玲	男	2005-10-21	20230505
20230505003	吴涛	女	2005-11-29	20230505
20230505004	赵联锋	女	2005-09-12	20230505
20230505005	王渺	女	2004-10-19	20230505

ALTER TABLE CLASSINFO ADD

-- 班级人数更新触发器

DELIMITER //

CREATE TRIGGER T_StudInfo_INSERT

AFTER INSERT

ON StudInfo

FOR EACH ROW

BEGIN

UPDATE ClassInfo

SET StudCount=StudCount+1

WHERE ClassID=NEW.ClassID;

END //

DELIMITER ;

INSERT INTO StudInfo VALUES

(**'20231152100'****,****'孙悟空'****,****'男'****,****NULL****,**
'20230712');



思考 & 作业

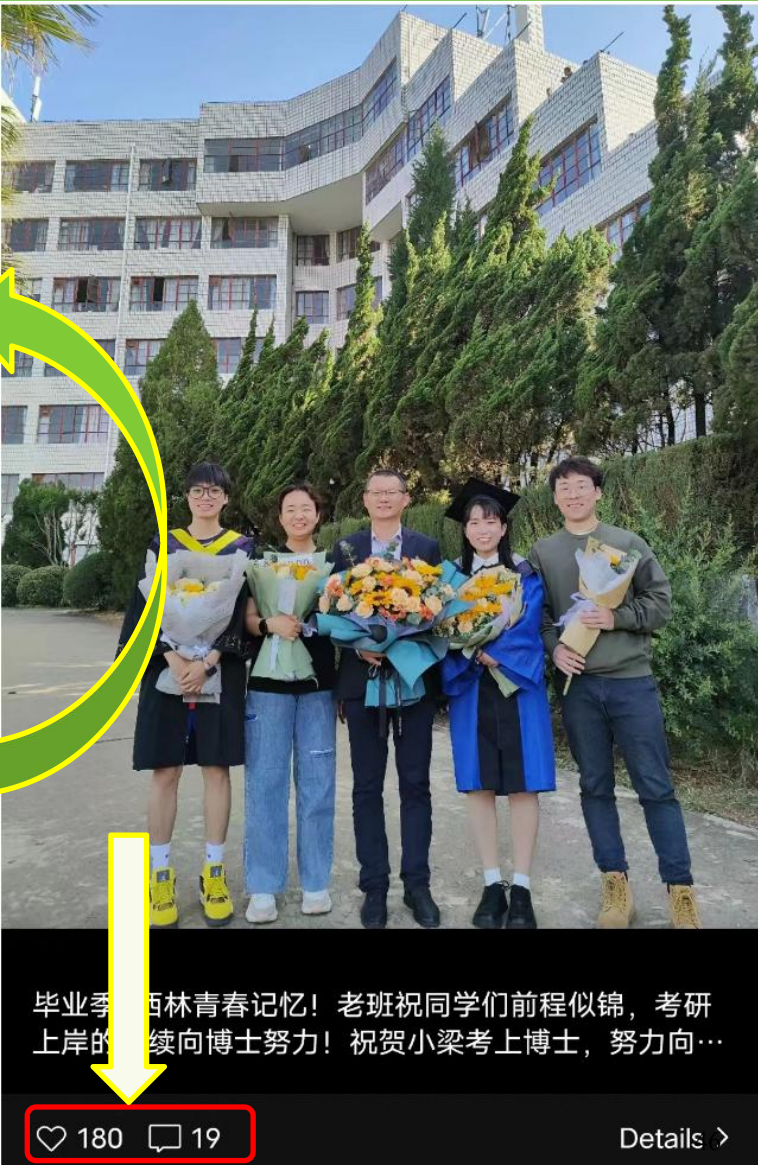
- 朋友圈的触发器原理

Post

PostID	WechatID	Content	PostTime	Likes	Comments
1001	NingLu	Good	2023-05-17	180	19
2005	...	...	...	...	...

Comments

CommentsID	PostID	WechatID	Content	CMTime
5005	1001	Tom	Good	2023-05-17
5007	1001	John	Wonderfull	2023-05-18
...	...	...	...	...





10.3 游标

- 可以把**Cursor**看成是一个用来保存“数据集”（多条记录）的对象
- 因**SELECT**语句查询的结果，是直接返回前端应用程序中，而无法在**SQL**程序中一笔一笔地处理
- 可以将挑选出来的结果先放入**Cursor**中，然后利用循环将每一条记录从**Cursor**中取出来处理



使用游标

- 声明
 - declare 游标名 cursor for select_statement
- 打开游标
 - open 游标名
- 从游标中取值
 - fetch 游标名 into var1,var2[,...] --将取到的一行赋值给多个变量
- 关闭游标
 - close 游标名



游标示例

```
create procedure Proc_cur_studinfo()
begin
  declare SID varchar(30) ;
  declare SName varchar(30);
  declare SGender varchar(10);
      declare have int default 1;
  declare cur_studinfo cursor for
      select studno,studname,studgender from studinfo;
  declare exit handler for NOT FOUND set have:= 0;
      open cur_studinfo;
      repeat
          fetch cur_studinfo into SID,SName,SGender;
          select SID,SName;
      until have = 0 end repeat;
  close cur_studinfo;
end;
```



学生成绩排名游标示例——创建平均分视图

Create View V_StudAvgScore

as

Select S.StudNo,StudName,

Cast(Avg(StudScore) AS Decimal(4,1))

As AvgScore

From StudInfo S,StudScoreInfo SI

Where S.StudNo=SI.StudNo

Group By S.StudNo,StudName;



学生成绩排名游标示例——存储过程、游标

```
Create Procedure P_GetStudQuene()
begin
Declare SNo varchar(20);
Declare SName varchar(20);
Declare i int default 1;
Declare Avg_Score decimal(4,1);
declare have int default 1;
Declare Cur_StudQuene Cursor For Select StudNo,StudName,AvgScore From V_StudAvgScore Order By
AvgScore desc;
declare exit handler for NOT FOUND set have:= 0;
Open Cur_StudQuene;
repeat
Fetch Cur_StudQuene into SNO,SName,Avg_Score;
select SNO,SName,Avg_Score,i;
set i=i+1;
until have = 0 end repeat;
Close Cur_StudQuene;
end;
```



成绩同名处理示例

```
Create Procedure P_GetStud_same_Quene()  
begin  
Declare SNo varchar(20);  
Declare SName varchar(20);  
Declare i int default 1;  
Declare j int default 1;  
Declare Avg_Score decimal(4,1);  
Declare prev_j int default 1;  
Declare prev_score decimal(4,1);  
declare have int default 1;  
Declare Cur_StudQuene Cursor For Select StudNo,StudName,AvgScore From  
V_StudAvgScore Order By AvgScore desc;  
declare exit handler for NOT FOUND set have:= 0;
```



成绩同名处理示例

```
Open Cur_StudQuene;  
repeat  
  Fetch Cur_StudQuene into SNO,SName,Avg_Score;  
    if avg_score=prev_score then  
      set j=prev_j;  
    else  
      set j=i;  
    end if;  
  select SNO,SName,Avg_Score,j;  
    set prev_j=j;  
    set prev_score=avg_score;  
  
  set i=i+1;  
until have = 0 end repeat;  
Close Cur_StudQuene;  
end;
```



成绩同名处理示例——方法二

DELIMITER //

DROP PROCEDURE IF EXISTS P_GetQuence;

Create Procedure P_GetQuence()

begin

set @strsql= 'Drop Table if Exists Temp_Quence;';

PREPARE drop_sql from @strsql;

execute drop_sql;

set @strsql=" Create Table Temp_Quence(

StudNo Varchar(15),

StudName Varchar(20),

AvgScore Numeric(4,1),

QuenceOne int auto_increment primary key,

QuenceTwo int null);";

Prepare create_sql from @strsql;

execute create_sql;



成绩同名处理示例——方法二

```
Insert Into Temp_Quence(StudNo,StudName,AvgScore)  
  Select StudNo,StudName,AvgScore  
  From V_StudAvgScore  
  Order By AvgScore Desc;  
      Update Temp_Quence,  
      (Select AvgScore,Min(QuenceOne) MinQuence  
      From Temp_Quence Group By AvgScore) A  
      Set QuenceTwo=A.MinQuence  
      Where Temp_Quence.AvgScore=A.AvgScore;  
  
end  
//  
DELIMITER;
```



成绩系统：学生成绩同名次处理 (RANK())

```
SELECT StudNo,  
       CAST(AVG(StudScore) AS DECIMAL(4,1)) AS AvgScore,  
       RANK() OVER (ORDER BY CAST(AVG(StudScore) AS DECIMAL(4,1)) DESC) AS Rank_Num,  
       DENSE_RANK() OVER (ORDER BY CAST(AVG(StudScore) AS DECIMAL(4,1)) DESC) AS  
Rank_Dense  
FROM StudScoreInfo  
GROUP BY StudNo;
```



StudNo	AvgScore	Rank_Num	Rank_Dense
20231151008	84.2	12	12
20230711055	84.1	13	13
20230716003	84.0	14	14
20231152058	84.0	14	14
20230711005	83.9	16	15
20231151043	83.9	16	15
20231152016	83.9	16	15
20230715026	83.7	19	16



下次课内容

- 11.1 用户管理
- 11.2 权限管理
- 11.3 角色管理
- 11.4 批量用户管理